

JAMSEC*Servicios Gestionados de Ciberseguridad***INFORME TÉCNICO DE AUDITORÍA DE SEGURIDAD**

Pruebas de intrusión sobre la infraestructura tecnológica

Cliente: Apex Gestoría

Referencia	JAMSEC-AUD-2026-001
Versión	1.0
Fecha	Junio de 2026
Autor	Alex Sánchez García — JAMSEC
Clasificación	CONFIDENCIAL

Este documento es confidencial. No debe compartirse con terceros sin permiso de Apex Gestoría y JAMSEC.

Índice1. Resumen ejecutivo

Apex Gestoría nos encargó comprobar hasta dónde podría llegar un atacante que se propusiera entrar en sus sistemas. Para responder a esa pregunta hicimos una auditoría de intrusión, es decir, atacamos su infraestructura igual que lo haría alguien con malas intenciones, pero de forma controlada y dejando constancia de cada paso.

El resultado no es bueno y conviene decirlo claro: conseguimos hacernos con el control completo de los sistemas y llegar hasta los datos más sensibles del negocio (información de clientes y nóminas de empleados), partiendo de cero y desde internet. En total encontramos 9 problemas de seguridad, de los cuales 3 son de gravedad crítica y 4 alta.

Lo más preocupante no es ningún fallo aislado, sino la facilidad con la que se encadenan: aprovechamos una web vulnerable, un servidor de ficheros mal configurado que filtraba contraseñas y una separación de redes demasiado permisiva para ir saltando de un sistema a otro hasta el final. Un atacante con conocimientos medios podría repetir esto sin demasiado esfuerzo.

La parte positiva es que todos los problemas tienen solución y la mayoría son de configuración, no requieren grandes inversiones. En el apartado 8 proponemos un plan de corrección ordenado por prioridad, empezando por lo más urgente.

1.1. Cuadro de severidades

Crítica	Alta	Media	Baja
3	4	2	0

2. Alcance y objetivos

Acordamos con el cliente revisar tanto lo que da la cara a internet como lo que hay por detrás una vez se consigue entrar. En concreto, el alcance incluyó el punto de acceso de Apex (172.20.17.230) y los sistemas internos a los que se pudiera llegar a partir de ahí. Trabajamos en modo «caja negra»: empezamos sin ninguna información ni credencial, igual que un atacante real.

2.1. Activos en alcance

Activo	Dirección	Función
apex-fw	172.20.17.230 (WAN)	Cortafuegos perimetral
apex-web	10.30.20.10 (DMZ)	Servidor web público
apex-db	10.30.10.10 (LAN)	Base de datos MariaDB
apex-srv	10.30.10.20 (LAN)	Servidor interno / FTP

2.2. Objetivos

- Encontrar las vulnerabilidades explotables de los servicios que están expuestos.
- Comprobar el impacto real de cada una explotándola de forma controlada.
- Ver hasta qué datos del cliente se podría llegar.
- Comprobar si el sistema de monitorización detecta el ataque.
- Entregar recomendaciones de corrección ordenadas por prioridad.

3. Metodología

Para que la auditoría fuera ordenada y no se nos escapara nada, seguimos el marco PTES (Penetration Testing Execution Standard), que divide el trabajo en fases. Para la parte web nos

apoyamos además en el OWASP Top 10 de 2021, que es la referencia de los fallos web más habituales. Y para puntuar la gravedad de cada hallazgo usamos CVSS v3.1, el estándar que asigna a cada problema una nota del 0 al 10.

3.1. Fases del trabajo

- Recopilación de información: ver qué hay expuesto y por dónde se puede entrar.
- Análisis de vulnerabilidades: identificar qué fallos tiene cada servicio.
- Explotación: aprovechar esos fallos de forma controlada.
- Post-explotación: una vez dentro, ver hasta dónde se puede llegar (movimiento lateral).
- Documentación: recoger las pruebas y redactar este informe.

3.2. Cómo interpretar la severidad (CVSS v3.1)

Severidad	Rango CVSS	Cuándo actuar
Crítica	9.0 – 10.0	Hay que corregirlo de inmediato
Alta	7.0 – 8.9	Corregir cuanto antes (días)
Media	4.0 – 6.9	Planificar la corrección
Baja	0.1 – 3.9	Corregir cuando se pueda

3.3. Herramientas utilizadas

Para el reconocimiento de red usamos nmap; para la parte web, sqlmap y whatweb; para probar contraseñas, hydra; y para el acceso y el movimiento por dentro, clientes estándar de SSH, FTP y MariaDB. Todo se lanzó desde la estación de auditoría de JAMSEC, una máquina Debian con las herramientas habituales de pentesting.

4. Resumen de hallazgos

Esta tabla resume los nueve problemas encontrados. Cada uno se explica en detalle en el apartado siguiente.

JAM-01	Inyección SQL en el formulario de acceso (login.php)	Crítica	9.8
JAM-02	Base de datos MariaDB expuesta con superusuario remoto y contraseña débil	Crítica	9.8
JAM-03	Subida de ficheros sin validación (ejecución remota de código)	Crítica	9.8
JAM-04	Credenciales SSH débiles susceptibles de fuerza bruta	Alta	8.8
JAM-05	Servidor FTP con acceso anónimo y exposición de información sensible	Alta	8.2
JAM-06	Segmentación de red deficiente: la DMZ accede a la red interna (LAN)	Alta	7.4
JAM-07	Almacenamiento de contraseñas en texto claro	Alta	7.5
JAM-08	Comunicaciones sin cifrar (HTTP y FTP en texto claro)	Media	5.9
JAM-09	Interfaz de administración SSH del cortafuegos expuesta	Media	5.3

5. Hallazgos detallados

JAM-01 — Inyección SQL en el formulario de acceso (login.php)

Severidad Crítica (CVSS 9.8)

Vector CVSS AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Categoría OWASP

A03:2021 – Injection

Activos afectados

apex-web (10.30.20.10) — http://172.20.17.230/login.php

En qué consiste

El formulario de acceso de la web de clientes coge el usuario que se escribe y lo pega directamente dentro de una consulta SQL, sin separarlo de la consulta ni filtrar lo que llega. Eso permite dos cosas: por un lado, entrar sin saber la contraseña (escribiendo algo como admin' OR '1'='1'); y por otro, y mucho más grave, ir «preguntándole» a la base de datos por su contenido a través del propio formulario, hasta sacarlo entero.

Cómo lo comprobamos

Lanzamos sqlmap contra el parámetro del usuario. Confirmó que era vulnerable por tres vías distintas (booleana, por tiempo y por UNION) y, a partir de ahí, le pedimos que volcara las tablas. Sin tener ninguna credencial válida, acabamos con las tablas de clientes, nóminas y usuarios delante.

Evidencia

Ficheros 03-sqlmap-dbs.txt y 04-sqlmap-dump.txt: muestran las inyecciones detectadas y el volcado completo de la base de datos apexgestoria.

Por qué es un problema

Es el fallo más grave de toda la auditoría. Deja al descubierto datos personales y bancarios de clientes y empleados (NIF, DNI, IBAN, salarios) y las contraseñas de la propia aplicación. Más allá del robo de información, supone un incumplimiento directo del RGPD, con las sanciones y el daño de imagen que eso conlleva para una gestoría, que precisamente vive de la confianza con esos datos.

Cómo solucionarlo

La solución de fondo es usar consultas parametrizadas (prepared statements con PDO o mysqli), que separan la consulta de los datos y hacen imposible este ataque. Además conviene validar lo que llega del formulario, dar a la aplicación un usuario de base de datos con los mínimos permisos (no uno que lo pueda ver todo) y, como red de seguridad mientras se corrige el código, poner un WAF delante.

JAM-02 — Base de datos MariaDB expuesta con superusuario remoto y contraseña débil

Severidad

Crítica (CVSS 9.8)

Vector CVSS	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Categoría OWASP	A05:2021 – Security Misconfiguration
Activos afectados	apex-db (10.30.10.10:3306)

En qué consiste

La base de datos no está «escondida» dentro de la red: escucha en todas las interfaces y, encima, deja que el usuario root (el administrador, que lo puede todo) se conecte desde cualquier sitio usando la contraseña 'toor', que es de las primeras que prueba cualquiera. Es la combinación de dos errores que por separado ya son graves: exponer el servicio y protegerlo con una contraseña trivial.

Cómo lo comprobamos

La contraseña la sacamos antes, del fichero que estaba colgado en el FTP (ver JAM-05). Con ella, durante el pivote, nos conectamos como root a la base de datos interna y leímos la tabla de nóminas sin ningún problema.

Evidencia

Fichero 07-pivote.txt: registra el acceso root a MariaDB y el volcado de la tabla de nóminas desde la red interna.

Por qué es un problema

Quien llegue a la base de datos con root puede leerlo todo, pero también modificarlo o borrarlo. Para una gestoría eso significa desde el robo de información de clientes hasta la posibilidad de falsear datos contables o, en el peor caso, destruir la información.

Cómo solucionarlo

Hacer que MariaDB escuche solo en la red interna (bind-address), eliminar el acceso de root desde fuera ('root'@'%'), poner contraseñas largas y distintas para cada servicio, y permitir conexiones únicamente desde el servidor de la aplicación. Activar también el cifrado de la conexión.

JAM-03 — Subida de ficheros sin validación (ejecución remota de código)

Severidad	Crítica (CVSS 9.8)
Vector CVSS	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H

Categoría OWASP

A04:2021 – Insecure Design

Activos afectados

apex-web (10.30.20.10) — panel.php

En qué consiste

Una vez dentro del panel, la web deja subir cualquier fichero sin mirar ni el tipo ni la extensión, y lo guarda en una carpeta a la que se llega desde el navegador. Eso quiere decir que se puede subir un fichero PHP malicioso (una «webshell») y luego abrirlo desde el navegador para ejecutar comandos en el servidor. Como el acceso al panel ya lo conseguimos saltándonos el login (JAM-01), la barrera de «estar autenticado» no protege nada.

Cómo lo comprobamos

Revisamos el código del panel: usa la función de subida directamente sobre el nombre original del fichero, sin comprobaciones, y la carpeta de destino tiene permisos de escritura. Con eso, subir una webshell y ejecutarla es inmediato.

Evidencia

Código de panel.php y carpeta /uploads con permisos abiertos. Encadenado con JAM-01.

Por qué es un problema

Pasar de «robar datos» a «controlar el servidor». Con ejecución de comandos en la máquina web, esa máquina se convierte en el trampolín para entrar en el resto de la red, que es justo lo que pasó en este caso.

Cómo solucionarlo

Aceptar solo los tipos de fichero que de verdad hagan falta (lista blanca de extensiones y comprobando el contenido), cambiarles el nombre al guardarlos, guardarlos fuera de la carpeta pública y, sobre todo, impedir que se ejecute PHP en la carpeta de subidas.

JAM-04 — Credenciales SSH débiles susceptibles de fuerza bruta**Severidad**

Alta (CVSS 8.8)

Vector CVSS

AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L

Categoría OWASP

A07:2021 – Identification and Authentication Failures

Activos afectados

apex-web (SSH publicado en 172.20.17.230:2222)

En qué consiste

El acceso por SSH del servidor web está abierto a internet y existe una cuenta, 'soporte', con la contraseña 'soporte123'. Es una contraseña corta, predecible y que aparece en cualquier diccionario, así que adivinarla es cuestión de minutos con una herramienta automática.

Cómo lo comprobamos

Con hydra y una lista pequeña de contraseñas habituales sacamos la pareja soporte / soporte123 en muy pocos intentos. A partir de ahí teníamos una sesión real en el servidor.

Evidencia

Fichero 06-hydra-ssh.txt: muestra la credencial encontrada.

Por qué es un problema

Con ese usuario entramos en la máquina de la DMZ y, como además tenía permisos de administrador (sudo), desde ahí pudimos movernos hacia la red interna. Fue una de las piezas clave de la cadena.

Cómo solucionarlo

Lo ideal es desactivar el acceso por contraseña y permitir solo clave pública. Si no, como mínimo: contraseñas robustas, bloqueo tras varios intentos fallidos (fail2ban) y no exponer el SSH directamente a internet, sino a través de una VPN o un bastión.

JAM-05 — Servidor FTP con acceso anónimo y exposición de información sensible**Severidad**

Alta (CVSS 8.2)

Vector CVSS

AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

Categoría OWASP

A05:2021 – Security Misconfiguration

Activos afectados

apex-srv (FTP publicado en 172.20.17.230:21)

En qué consiste

El servidor FTP permite entrar sin usuario ni contraseña (acceso anónimo) y, además, dejar ficheros. Por si fuera poco, dentro hay un fichero de texto con credenciales internas y una

copia de la base de datos. Es decir, cualquiera de internet puede entrar y descargarse información que debería estar muy protegida.

Cómo lo comprobamos

Entramos como anónimo y nos descargamos los ficheros que había. Uno de ellos, credentials.txt, contenía la contraseña de la base de datos (root/toor), la de la aplicación y la del router.

Evidencia

Ficheros 05-ftp.txt y ftp-credentials.txt: registran el acceso anónimo y el contenido descargado.

Por qué es un problema

Este fallo es el que abre la puerta a JAM-02: las contraseñas que se filtran aquí son las que luego usamos para acceder a la base de datos. Además, poder dejar ficheros de forma anónima permitiría a un atacante alojar contenido malicioso en el servidor.

Cómo solucionarlo

Desactivar el acceso anónimo, sacar de ahí cualquier fichero con información sensible, obligar a usar FTPS o SFTP (cifrado) y limitar el acceso a las IP que de verdad lo necesiten.

JAM-06 — Segmentación de red deficiente: la DMZ accede a la red interna (LAN)

Severidad	Alta (CVSS 7.4)
Vector CVSS	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
Categoría OWASP	A01:2021 – Broken Access Control
Activos afectados	apex-fw (política de cortafuegos)

En qué consiste

La DMZ es la zona donde se ponen los servicios que dan la cara a internet, precisamente porque son los más expuestos. La idea es que, si caen, el atacante quede «encerrado» ahí y no pueda llegar a la red interna. Aquí eso no se cumple: el cortafuegos deja que la DMZ hable libremente con la red interna, así que comprometer la web equivale a tener vía libre hacia la base de datos y el servidor de ficheros.

Cómo lo comprobamos

Una vez dentro del servidor web (DMZ), comprobamos que llegábamos sin problemas a la base de datos (10.30.10.10:3306) y al servidor de ficheros (10.30.10.20) de la red interna.

Evidencia

Fichero 07-pivote.txt: muestra el alcance desde la DMZ hacia los servicios de la LAN.

Por qué es un problema

Convierte un problema «de escaparate» (la web) en un problema «de trastienda» (los datos internos). Es lo que nos permitió el salto final hasta las nóminas.

Cómo solucionarlo

Apretar las reglas del cortafuegos para que desde la DMZ solo se pueda llegar a lo justo y necesario (por ejemplo, el puerto de la base de datos que use la web, y nada más), denegando todo lo demás por defecto.

JAM-07 — Almacenamiento de contraseñas en texto claro

Severidad	Alta (CVSS 7.5)
Vector CVSS	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Categoría OWASP	A02:2021 – Cryptographic Failures
Activos afectados	apex-db — tabla apexgestoria.usuarios

En qué consiste

Las contraseñas de los usuarios de la aplicación están guardadas tal cual, en texto legible. Lo correcto es guardar solo una versión «picada» (un hash) de la que no se puede volver atrás. Al estar en claro, basta con poder leer la tabla para tener las contraseñas de todos.

Cómo lo comprobamos

Al volcar la tabla usuarios con la inyección SQL, las contraseñas aparecieron directamente legibles (por ejemplo admin / Apex.2026!).

Evidencia

Fichero 04-sqlmap-dump.txt: muestra la tabla de usuarios con las contraseñas visibles.

Por qué es un problema

Cualquier acceso a la base de datos entrega de regalo las contraseñas, que además la gente suele reutilizar en otros servicios (correo, banca, etc.), ampliando el daño mucho más allá de

esta aplicación.

Cómo solucionarlo

Guardar las contraseñas con un algoritmo de hash pensado para ello y con sal (bcrypt o Argon2) y forzar el cambio de todas las que se hayan visto comprometidas.

JAM-08 — Comunicaciones sin cifrar (HTTP y FTP en texto claro)

Severidad	Media (CVSS 5.9)
Vector CVSS	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N
Categoría OWASP	A02:2021 – Cryptographic Failures
Activos afectados	apex-web (HTTP/80), apex-srv (FTP/21)

En qué consiste

Ni la web ni el FTP usan cifrado: la web va por HTTP (no HTTPS) y el FTP es el clásico sin TLS. Eso significa que las contraseñas y los datos viajan en texto plano por la red, donde alguien que esté «escuchando» podría leerlos.

Cómo lo comprobamos

El escaneo mostró el puerto 80 abierto y el 443 (HTTPS) cerrado, y el FTP sin opción de cifrado.

Evidencia

Fichero 01-nmap-servicios.txt.

Por qué es un problema

En una red que no controlas (una WiFi, por ejemplo), un atacante podría capturar credenciales y datos de clientes simplemente observando el tráfico, sin necesidad de «romper» nada.

Cómo solucionarlo

Poner HTTPS con un certificado válido y forzar la redirección de HTTP a HTTPS; sustituir el FTP por SFTP o, como mínimo, activar FTPS.

JAM-09 — Interfaz de administración SSH del cortafuegos expuesta

Severidad	Media (CVSS 5.3)
Vector CVSS	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Categoría OWASP	A05:2021 – Security Misconfiguration
Activos afectados	apex-fw (172.20.17.230:22)

En qué consiste

El SSH con el que se administra el propio cortafuegos es accesible desde la red de tránsito. El cortafuegos es el aparato que protege todo lo demás, así que tener su administración expuesta amplía la superficie de ataque justo donde menos interesa.

Cómo lo comprobamos

El escaneo con nmap detectó el puerto 22 abierto en el perímetro, además de los servicios publicados.

Evidencia

Fichero 01-nmap-servicios.txt.

Por qué es un problema

Abre la puerta a ataques de fuerza bruta o a intentar aprovechar alguna vulnerabilidad contra el propio dispositivo de seguridad.

Cómo solucionarlo

Permitir la administración solo desde una red de gestión dedicada o por VPN, usar autenticación por clave con doble factor y filtrar por IP de origen.

6. Cadena de ataque

Cada fallo por separado ya es un problema, pero lo de verdad grave es cómo se encadenan. Esta es la ruta que seguimos para pasar de no tener nada a controlar la base de datos interna:

- Primero, un escaneo para ver qué servicios tenía Apex expuestos (web, SSH y FTP).
- Entramos por FTP de forma anónima y nos descargamos un fichero con las contraseñas internas (JAM-05).
- Atacamos la web con inyección SQL y nos llevamos toda la base de datos por el formulario de login (JAM-01).

- Por fuerza bruta sacamos el usuario 'soporte' del SSH del servidor web (JAM-04).
- Desde ese servidor, y gracias a que la DMZ podía hablar con la red interna, saltamos hacia dentro (JAM-06).
- Con la contraseña que habíamos sacado del FTP, entramos como administrador en la base de datos interna y nos llevamos las nóminas (JAM-02).

Resumiendo: empezamos fuera, sin nada, y terminamos con los datos más sensibles del negocio. Y, como se ve, bastaba con haber tenido bien una sola de las capas (el FTP, la web o la separación de redes) para cortar la cadena.

7. Detección y monitorización

Como parte del servicio, JAMSEC tiene desplegado un sistema de monitorización (Wazuh como SIEM y Suricata como IDS de red). Durante la auditoría comprobamos que sí registró buena parte de la actividad del ataque, lo cual es una buena noticia: significa que, con el alertado bien configurado, este tipo de intrusión se puede detectar a tiempo.

En concreto, quedaron registrados:

- El escaneo de puertos inicial, detectado por Suricata.
- Los intentos fallidos de adivinar la contraseña por SSH y el acceso que finalmente funcionó.
- El acceso anónimo al servidor FTP.
- Los errores que iba devolviendo la web mientras la atacábamos con inyección SQL.

La recomendación aquí es dar el siguiente paso: configurar avisos automáticos para estos patrones (que alguien se entere en el momento, no a posteriori) y valorar poner el IDS en modo de bloqueo activo en el perímetro.

8. Conclusiones y plan de remediación

El estado de seguridad actual de Apex es insuficiente: con poco esfuerzo se llega a los datos más críticos. Ahora bien, la mayoría de los problemas son de configuración y se pueden arreglar sin grandes inversiones. Proponemos abordarlos en este orden:

Prioridad	Acción	Hallazgos
Inmediata	Corregir la inyección SQL de la web (consultas parametrizadas) y cambiar todas las contraseñas	JAM-01, JAM-07
Inmediata	Sacar la base de datos de internet y quitarle el root remoto con contraseña débil	JAM-02

Inmediata	Arreglar la subida de ficheros y no dejar ejecutar PHP en la carpeta de subidas	JAM-03
Urgente	Reforzar el SSH (solo clave, fail2ban) y no exponerlo a internet	JAM-04, JAM-09
Urgente	Desactivar el FTP anónimo y retirar los ficheros sensibles	JAM-05
Urgente	Apretar la separación entre la DMZ y la red interna	JAM-06
Planificada	Poner HTTPS y FTPS en todos los servicios	JAM-08

Una vez aplicadas estas medidas, recomendamos repetir la auditoría para confirmar que los fallos se han cerrado de verdad y que no han aparecido otros nuevos.

9. Anexos

Las pruebas completas de cada hallazgo (las salidas de las herramientas) se entregan junto con este informe, en la carpeta de evidencias del repositorio del proyecto:

- 01-nmap-servicios.txt — reconocimiento de servicios.
- 03-sqlmap-dbs.txt / 04-sqlmap-dump.txt — inyección SQL y volcado de datos.
- 05-ftp.txt / ftp-credentials.txt — acceso FTP anónimo y ficheros descargados.
- 06-hydra-ssh.txt — fuerza bruta de SSH.
- 07-pivote.txt — movimiento de la DMZ a la red interna y robo de las nóminas.